

Lab

Name: Arden Diago

Reg No:

Class: MCA-A

Aim

To collect real-world student survey data using Google Forms, preprocess and clean the dataset
Simple Linear Regression using Scikit-learn, manually compute the regression equation using O
Least Squares (OLS) formulas, compare both approaches, and save the learned model paramet

```
In [65]: import pandas as pd
import numpy as nu
from matplotlib import pyplot as plt
```

Part A: Data Collection and Preprocessing

Perform the following steps:

- Load the dataset using Pandas
- Display the first rows
- Check dataset dimensions
- Identify missing values
- Remove or handle null values
- Convert required columns into numerical datatype
- Remove duplicate records if present
- Generate statistical summary
- Select appropriate dependent and independent variables

```
In [66]: # Data Path
DATA = "reports.csv"

df = pd.read_csv(DATA)

# The Google Form export has stray whitespace/newlines in some headers
# (e.g. "Registration Number ", "...interested in\n"), which breaks exact
# column-name lookups later on, so normalize them right after loading.
df.columns = df.columns.str.strip().str.replace('\n', '', regex=False)

df.head()
```

Out[66]:

	Timestamp	Registration Number	Email	Job that interests
0	4 32 2024 4 32 3	250712	jyaelza.jabi@mca.christuniversity.in	Software Engineer
0	4 32 2024 4 32 37	250712	jinishaleema.rosario@mca.christuniversity.in	Engineer
0	4 32 2024 4 32 45	250712	rajeev.chandar@mca.christuniversity.in	Software Engineer
0	4 32 2024 4 32 51	250712	sounak.chakraborty@mca.christuniversity.in	Scientist
0	4 32 2024 4 32 55	250712	samar.subhash@mca.christuniversity.in	Software Engineer

```
In [67]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 54 entries, 0 to 53
Data columns (total 15 columns):
 #
Column      Non-Null Count  Dtype
---
-----
0
Timestamp    54 non-null    str
1  Registration
Number
non-null     str
2
Email
54 non-null    str
3  Job role that you are interested
in
54 non-null    st
4  What is the minimum salary of students placed through campus (In LPA..re
as a number) 54 non-null    str
5  What is the maximum salary of students placed through campus (In LPA..re
as a number) 54 non-null    str
6  What is the median salary of students placed through campus (In LPA..re
as a number) 54 non-null    str
7  Which is the highest paying
company
54 non-null
str
8  Rate your contribution towards extra curricular
activities    54 non-null    int64
9  Rate your technical
competencies
54 no
null         int64
10 What are your package expectations
(LPA)
54 non-null    str
11 your CIA % of last
semester
54 n
null         str
12 your GPA of last
semester
54
null         str
13 Your maximum attendance % till last
semester
54 non-null    str
14 Internships
Interests
non-null     str
dtypes: int64(2), str(13)
memory usage: 6.5 KB

```

In [68]: `df.shape`

Out[68]: (54, 15)

This Dataset has Rows and Columns

In [69]: `df.isnull().sum()`

```

Out[69]: Timestamp
          0
Registration
Number
Email
          0
Job role that you are interested
in
          0
What is the minimum salary of students placed through campus (In LPA..respon
number)    0
What is the maximum salary of students placed through campus (In LPA..respon
number)    0
What is the median salary of students placed through campus (In LPA..respon
number)    0
Which is the highest paying
company
          0
Rate your contribution towards extra curricular
activities
          0
Rate your technical
competencies
          0
What are your package expectations
(LPA)
          0
your CIA % of last
semester
          0
your GPA of last
semester
          0
Your maximum attendance % till last
semester
          0
Internships
Interests
dtype: int64

```

`isnull()` reports missing values, but that's misleading. Several columns contain placeholder text instead of a real answer — e.g. `Option 1` (a leftover default from the Google Form), `na`, etc. These behave like missing values and only become visible once we clean and convert the columns to numeric below.

```
In [70]: df.columns
```

```

Out[70]: Index(['Timestamp', 'Registration Number', 'Email',
               'Job role that you are interested in',
               'What is the minimum salary of students placed through campus (In
LPA..respond as a number)',
               'What is the maximum salary of students placed through campus (In
LPA..respond as a number)',
               'What is the median salary of students placed through campus (In
LPA..respond as a number)',
               'Which is the highest paying company',
               'Rate your contribution towards extra curricular activities',
               'Rate your technical competencies',
               'What are your package expectations (LPA)',
               'your CIA % of last semester', 'your GPA of last semester',
               'Your maximum attendance % till last semester',
               'Internships Interests'],
              dtype='str')

```

```
In [71]: len(df['Email'].unique())
```

Out[71]: 52

```
In [72]: placeholders = {"option 1", "na", "n/a", "-", "", "none"}

# Replace disguised missing values (Google Form placeholder text) with re
df = df.apply(lambda col: col.map(
    lambda v: nu.nan if isinstance(v, str) and v.strip().lower() in place
))

df.isnull().sum()
```

```
Out[72]: Timestamp
          0
Registration
Number
Email
          0
Job role that you are interested
in          0
What is the minimum salary of students placed through campus (In LPA..respor
number)    0
What is the maximum salary of students placed through campus (In LPA..respor
number)    9
What is the median salary of students placed through campus (In LPA..respor
number)    0
Which is the highest paying
company          0
Rate your contribution towards extra curricular
activities      0
Rate your technical
competencies          0
What are your package expectations
(LPA)          0
your CIA % of last
semester          1
your GPA of last
semester
Your maximum attendance % till last
semester        6
Internships
Interests
dtype: int64
```

```
In [73]: import re

def clean_numeric(value, is_salary=False):
    """Parse messy survey text (commas, %, LPA/cr suffixes, ranges like '
    if pd.isna(value):
        return nu.nan

    text = str(value).strip().lower()
    is_crore = bool(re.search(r'cr(ore)?\b', text))
    text = re.sub(r'lpa|lakh(s)?|cr(ore)?\b|%\b|rs\.\b|inr', '', text).repla

    numbers = [float(n) for n in re.findall(r'\d+\.\d*', text)]
    if not numbers:
        return nu.nan

    val = sum(numbers) / len(numbers) # average out ranges such as "8-12
```

```

    if is_crore:
        val *= 100 # 1 crore = 100 LPA
    if is_salary and val > 100:
        val /= 100000 # respondent answered in plain rupees instead of L

    return val

# Registration Number / Email are identifiers, not analysis variables, and
# responses even have a name typed into the registration-number field by
# mistake – so they're left alone. Only the salary, percentage and GPA co
# actually used for analysis need to be numeric.
salary_cols = [
    'What is the minimum salary of students placed through campus (In LPA
    'What is the maximum salary of students placed through campus (In LPA
    'What is the median salary of students placed through campus (In LPA
    'What are your package expectations (LPA)',
]

percent_cols = [
    'your CIA % of last semester',
    'Your maximum attendance % till last semester',
]

other_numeric_cols = ['your GPA of last semester']

for col in salary_cols:
    df[col] = df[col].apply(lambda v: clean_numeric(v, is_salary=True))

for col in percent_cols + other_numeric_cols:
    df[col] = df[col].apply(clean_numeric)

df[salary_cols + percent_cols + other_numeric_cols].dtypes

```

```

Out[73]: What is the minimum salary of students placed through campus (In LPA..respor
         number)      float64
         What is the maximum salary of students placed through campus (In LPA..respor
         number)      float64
         What is the median salary of students placed through campus (In LPA..respor
         number)      float64
         What are your package expectations
         (LPA)                                                float64
         your CIA % of last
         semester
         Your maximum attendance % till last
         semester                                                float64
         your GPA of last
         semester
         float64
         dtype: object

```

```

In [74]: # Re-check for missing values now that the columns are actually numeric
         df[salary_cols + percent_cols + other_numeric_cols].isnull().sum()

```

```
Out[74]: What is the minimum salary of students placed through campus (In LPA..respon  
number)      0  
What is the maximum salary of students placed through campus (In LPA..respon  
number)      9  
What is the median salary of students placed through campus (In LPA..respon  
number)      0  
What are your package expectations  
(LPA)                                                0  
your CIA % of last semester                                                    1  
Your maximum attendance % till last semester                                     6  
your GPA of last semester  
dtype: int64
```

```
In [75]: for col in salary_cols + percent_cols + other_numeric_cols:
          df[col] = df[col].fillna(df[col].median())

          df.isnull().sum()
```

Remove duplicate records if present

```
In [76]: df.duplicated().sum()
```

```
Out[76]: np.int64(0)
```

There are no exact full-row duplicates, but earlier `len(df['Email'].unique())` returned `52` against `51` rows — meaning `51` students submitted the form twice. These repeat submissions are removed below, keeping each student's most recent response.

```
In [77]: df = df.drop_duplicates(subset='Email', keep='last').reset_index(drop=True)
df.shape
```

```
Out[77]: (52, 15)
```

Generate statistical summary

```
In [78]: df.describe()
```

Out[78]:

	What is the minimum salary of students placed through campus (In LPA..respond as a number)	What is the maximum salary of students placed through campus (In LPA..respond as a number)	What is the median salary of students placed through campus (In LPA..respond as a number)	What is the contribution extra c
count	52	52	52	52
mean	1.5	1.5	1.5	1.5
std	1.5	1.5	1.5	1.5
min	1.5	1.5	1.5	1.5
%	1.5	1.5	1.5	1.5
%	1.5	1.5	1.5	1.5
%	1.5	1.5	1.5	1.5
max	1.5	1.5	1.5	1.5

The summary still shows a few implausible values inherited from the raw survey (e.g. a GPA `ma` on what should be a `-point` scale) — a couple of respondents appear to have swapped GPA and CIA% answers. These are left as-is rather than guessed at, but are worth keeping in mind as outliers going into Part B.

Select appropriate dependent and independent variable

Part B needs one independent variable (X) and one dependent variable (y) for Simple Linear Regression. The survey is centered on salary, so the dependent variable is **package expectation (LPA)**. Of 1

available predictors, GPA correlates with almost nothing here — but that's because a couple of e mixed up the GPA/CIA% fields (see above), not because GPA is genuinely unrelated. **CIA % of I semester** is the cleaner predictor of the two and is used as the independent variable: does a stu internal-assessment performance relate to how big a package they expect?

```
In [79]: independent_var = 'your CIA % of last semester'
dependent_var = 'What are your package expectations (LPA)'

X = df[[independent_var]]
y = df[dependent_var]

df[[independent_var, dependent_var]].corr()
```

```
Out[79]:
```

	your CIA % of last semester	What are your package expectations (LPA)
your CIA % of last semester	1	0.000000
What are your package expectations (LPA)	0.000000	1

Part B: Simple Linear Regression using Scikit-learn

Perform the following:

- Split the dataset into training and testing sets
- Train the model using `LinearRegression` from Scikit-learn
- Obtain:
 - Slope
 - Intercept
- Predict output values using the trained model

```
In [80]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

model = LinearRegression()
model.fit(X_train, y_train)
```

Out[80]:

▼ LinearRegression ⓘ		
Parameters		
	<code>fit_intercept</code>	True
	<code>copy_X</code>	True
	<code>tol</code>	1e-06
	<code>n_jobs</code>	None
	<code>positive</code>	False
Fitted attributes		
Name	Type	Value
<code>coef_</code>	ndarray[float64](1,)	[0.13]
<code>feature_names_in_</code>	ndarray[object](1,)	['your CIA % of last semester']
<code>intercept_</code>	float64	-0.4379
<code>n_features_in_</code>	int	1
<code>rank_</code>	int	1
<code>singular_</code>	ndarray[float64](1,)	[79.34]

```
In [81]: slope = model.coef_[0]
intercept = model.intercept_

print(f"Slope: {slope:.4f}")
print(f"Intercept: {intercept:.4f}")
```

Slope: 0.1319
Intercept: -0.4379

```
In [82]: y_pred_sklearn = model.predict(X_test)
y_pred_sklearn
```

```
Out[82]: array([ 8.39859074,  8.79425596,  9.84936322,  8.79425596,  9.71747481,
                  0.01708248,  9.84936322, 10.90447047,  8.79425596,  8.92614437,
                  11.30013569])
```

Part C: Manual Computation using Ordinary Least Squares (OLS)

Using the same training data as Part B, manually compute the regression line using the OLS formulae:

- **Slope:** $m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$
- **Intercept:** $c = \bar{y} - m\bar{x}$
- **Regression equation:** $\hat{y} = mx + c$

Construct the regression equation manually using the computed slope and intercept values.

```
In [83]: x_train_vals = X_train[independent_var].values
y_train_vals = y_train.values

x_mean = x_train_vals.mean()
y_mean = y_train_vals.mean()

slope_manual = ((x_train_vals - x_mean) * (y_train_vals - y_mean)).sum()
intercept_manual = y_mean - slope_manual * x_mean

print(f"Manual OLS slope: {slope_manual:.4f}")
print(f"Manual OLS intercept: {intercept_manual:.4f}")
```

Manual OLS slope: 0.1319

Manual OLS intercept: -0.4379

```
In [84]: y_pred_manual = slope_manual * X_test[independent_var].values + intercept_manual
y_pred_manual
```

```
Out[84]: array([ 8.39859074,  8.79425596,  9.84936322,  8.79425596,  9.71747481,
                0.01708248,  9.84936322, 10.90447047,  8.79425596,  8.92614437,
                11.30013569])
```

Comparison Task

Compare the following:

- . Predictions from Scikit-learn
- . Predictions from the manually computed OLS equation
- . Difference between both outputs

Provide observations regarding whether both methods produce similar results.

```
In [85]: comparison = pd.DataFrame({
    'Actual': y_test.values,
    'Sklearn Prediction': y_pred_sklearn,
    'Manual OLS Prediction': y_pred_manual,
    'Difference': y_pred_sklearn - y_pred_manual,
})
comparison
```

Out[85]:

	Actual	Sklearn Prediction	Manual OLS Prediction	Difference
0	0.0	0.000000	0.000000	0.000000
1	10.0	10.000000	10.000000	0.000000
2	10.0	10.000000	10.000000	0.000000
3	13.0	13.000000	13.000000	0.000000
4	40.0	40.000000	40.000000	0.000000
5	0.0	0.000000	0.000000	0.000000
6	0.0	0.000000	0.000000	0.000000
7	11.0	11.000000	11.000000	0.000000
8	0.0	0.000000	0.000000	0.000000
9	0.0	0.000000	0.000000	0.000000
10	0.0	0.000000	0.000000	0.000000
11	0.0	0.000000	0.000000	0.000000
12	0.0	0.000000	0.000000	0.000000
13	0.0	0.000000	0.000000	0.000000
14	0.0	0.000000	0.000000	0.000000
15	0.0	0.000000	0.000000	0.000000
16	0.0	0.000000	0.000000	0.000000
17	0.0	0.000000	0.000000	0.000000
18	0.0	0.000000	0.000000	0.000000
19	0.0	0.000000	0.000000	0.000000
20	0.0	0.000000	0.000000	0.000000
21	0.0	0.000000	0.000000	0.000000
22	0.0	0.000000	0.000000	0.000000
23	0.0	0.000000	0.000000	0.000000
24	0.0	0.000000	0.000000	0.000000
25	0.0	0.000000	0.000000	0.000000
26	0.0	0.000000	0.000000	0.000000
27	0.0	0.000000	0.000000	0.000000
28	0.0	0.000000	0.000000	0.000000
29	0.0	0.000000	0.000000	0.000000
30	0.0	0.000000	0.000000	0.000000
31	0.0	0.000000	0.000000	0.000000
32	0.0	0.000000	0.000000	0.000000
33	0.0	0.000000	0.000000	0.000000
34	0.0	0.000000	0.000000	0.000000
35	0.0	0.000000	0.000000	0.000000
36	0.0	0.000000	0.000000	0.000000
37	0.0	0.000000	0.000000	0.000000
38	0.0	0.000000	0.000000	0.000000
39	0.0	0.000000	0.000000	0.000000
40	0.0	0.000000	0.000000	0.000000
41	0.0	0.000000	0.000000	0.000000
42	0.0	0.000000	0.000000	0.000000
43	0.0	0.000000	0.000000	0.000000
44	0.0	0.000000	0.000000	0.000000
45	0.0	0.000000	0.000000	0.000000
46	0.0	0.000000	0.000000	0.000000
47	0.0	0.000000	0.000000	0.000000
48	0.0	0.000000	0.000000	0.000000
49	0.0	0.000000	0.000000	0.000000
50	0.0	0.000000	0.000000	0.000000
51	0.0	0.000000	0.000000	0.000000
52	0.0	0.000000	0.000000	0.000000
53	0.0	0.000000	0.000000	0.000000
54	0.0	0.000000	0.000000	0.000000
55	0.0	0.000000	0.000000	0.000000
56	0.0	0.000000	0.000000	0.000000
57	0.0	0.000000	0.000000	0.000000
58	0.0	0.000000	0.000000	0.000000
59	0.0	0.000000	0.000000	0.000000
60	0.0	0.000000	0.000000	0.000000
61	0.0	0.000000	0.000000	0.000000
62	0.0	0.000000	0.000000	0.000000
63	0.0	0.000000	0.000000	0.000000
64	0.0	0.000000	0.000000	0.000000
65	0.0	0.000000	0.000000	0.000000
66	0.0	0.000000	0.000000	0.000000
67	0.0	0.000000	0.000000	0.000000
68	0.0	0.000000	0.000000	0.000000
69	0.0	0.000000	0.000000	0.000000
70	0.0	0.000000	0.000000	0.000000
71	0.0	0.000000	0.000000	0.000000
72	0.0	0.000000	0.000000	0.000000
73	0.0	0.000000	0.000000	0.000000
74	0.0	0.000000	0.000000	0.000000
75	0.0	0.000000	0.000000	0.000000
76	0.0	0.000000	0.000000	0.000000
77	0.0	0.000000	0.000000	0.000000
78	0.0	0.000000	0.000000	0.000000
79	0.0	0.000000	0.000000	0.000000
80	0.0	0.000000	0.000000	0.000000
81	0.0	0.000000	0.000000	0.000000
82	0.0	0.000000	0.000000	0.000000
83	0.0	0.000000	0.000000	0.000000
84	0.0	0.000000	0.000000	0.000000
85	0.0	0.000000	0.000000	0.000000
86	0.0	0.000000	0.000000	0.000000
87	0.0	0.000000	0.000000	0.000000
88	0.0	0.000000	0.000000	0.000000
89	0.0	0.000000	0.000000	0.000000
90	0.0	0.000000	0.000000	0.000000
91	0.0	0.000000	0.000000	0.000000
92	0.0	0.000000	0.000000	0.000000
93	0.0	0.000000	0.000000	0.000000
94	0.0	0.000000	0.000000	0.000000
95	0.0	0.000000	0.000000	0.000000
96	0.0	0.000000	0.000000	0.000000
97	0.0	0.000000	0.000000	0.000000
98	0.0	0.000000	0.000000	0.000000
99	0.0	0.000000	0.000000	0.000000

Observation: the slope () and intercept () match exactly between learn and the manual OLS formulas, and every prediction differs by no more than $\sim 10^{-15}$ floating-point noise. This is expected: for a single-feature regression, Scikit-learn's `LinearRegression` solves the same closed-form OLS equations used manually here, so the approaches are mathematically equivalent, not just similar.

Note that the test set also contains one extreme outlier (actual = 1000000, predicted ≈ 1000000 — one of the joke/extreme "package expectation" responses identified back in Part A — which both methods predict identically poorly, since they share the same fitted line.

Parameter Saving Task

Save the parameters learned by the model using Pickle.

The following parameters must be saved:

- . Slope
- . Intercept

Suggested file name: `linear_regression_weights.pkl`

Demonstrate:

- . Saving parameters into a Pickle file
- . Loading the Pickle file
- . Using the loaded parameters for prediction

```
In [86]: import pickle

weights = {'slope': slope, 'intercept': intercept}
```

```
with open('linear_regression_weights.pkl', 'wb') as f:
    pickle.dump(weights, f)

with open('linear_regression_weights.pkl', 'rb') as f:
    loaded_weights = pickle.load(f)

loaded_weights
```

```
Out[86]: {'slope': np.float64(0.1318884069714909),
          'intercept': np.float64(-0.43793252678840666)}
```

```
In [87]: sample_x = X_test[independent_var].values[0]
         predicted = loaded_weights['slope'] * sample_x + loaded_weights['intercept']

         print(f"Input CIA%: {sample_x}")
         print(f"Predicted package expectation (LPA) using loaded parameters: {predicted}")
```

Input CIA%: 67.0

Predicted package expectation (LPA) using loaded parameters: 8.3986